

Problems

P₁.

a. src port: 123 dest port: 111

b. src port: 124 dest port: 111

c. src port: 111 dest port: 123

d. src port: 111 dest port: 124

e. Yes, because each host's OS decides port numbers for the running process, so there is a chance it may be same.

f. No, because each process is assigned a unique port number.

P₂.

src port: 80
80
80

dest port: 7532
26145
26145

ip src: B
B
B

ip dest: C
C
A

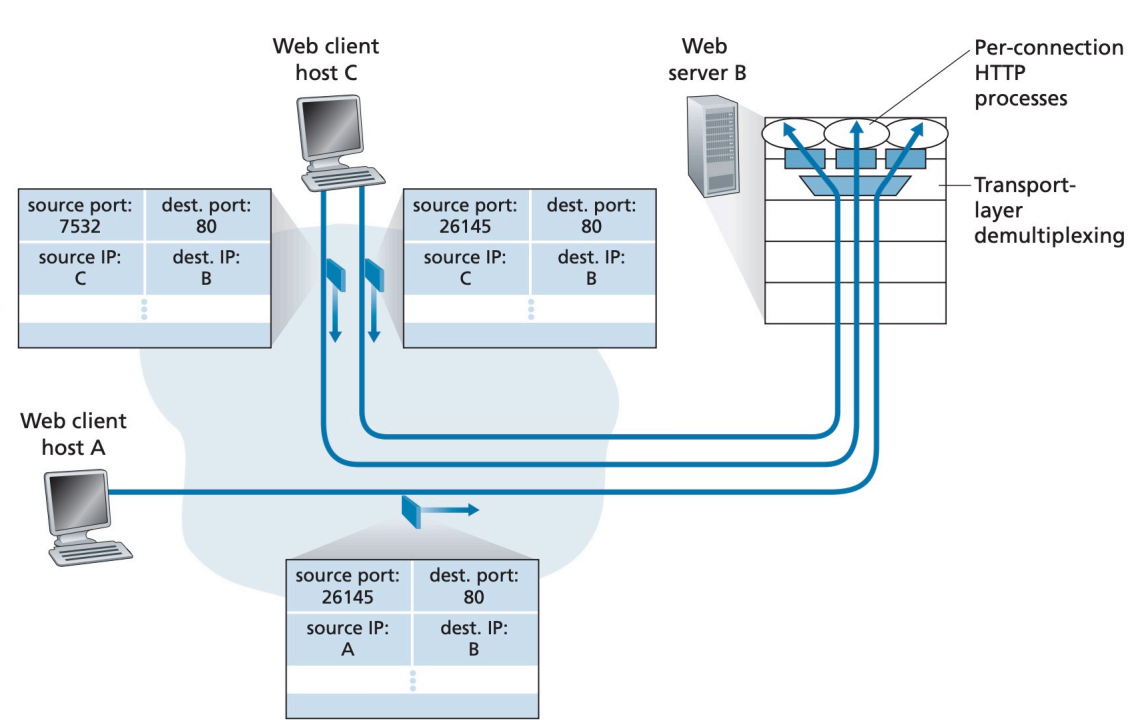


Figure 3.5 ♦ Two clients, using the same destination port number (80) to communicate with the same Web server application

P₃.

```
10 0 1 1 . 1 1
0 1 0 1 0 0 1 1
0 1 1 0 0 1 1 0
0 1 1 1 0 1 0 0
```

```
1 0 0 0 1 0 1 1 0 1
└─┬─┘
OF 0 0 1 0 1 1 1 0
1 1 0 1 0 0 0 1
```

UDP and TCP use the 1s complement because it simplifies the verification process at the receiver.

If the sender sends the sum itself, the receiver would have to calculate the sum of the data and then compare it to the received checksum value. By sending the **complement** of the sum, the receiver can simply sum all received words (data + checksum). If no errors occurred, the resulting 16-bit (or 8-bit in this case) value will be **all 1s**.

1-bit Error:

A 1-bit error will **always** be detected. If a single bit flips from 0 to 1 or 1 to 0, the total sum will change, ensuring that the final verification at the receiver will not result in all 1s.

2-bit Error:

It is **possible** for a 2-bit error to go undetected.

If one bit flips from 0 to 1 and another bit in the same column (or a mathematically related position) flips from 1 to 0, the total sum remains unchanged. Because the sum is the same, the receiver's check will still result in all 1s, even though the data is corrupted. This is a known limitation of the Internet Checksum compared to more robust methods like CRC (Cyclic Redundancy Check).

P₄.

(C)

If you flip **one bit from 0 to 1** in the first byte, you have added a specific value (2^n) to the total sum. To leave the checksum unchanged, you must flip **one bit from 1 to 0** in the second byte at the **exact same bit position**. This subtracts that same value (2^n), resulting in a net change of zero.

Original 8 bits:

- Byte 1: 0 1 0 1 1 1 0 0
- Byte 2: 0 1 1 0 0 1 0 1

The "Flip" Strategy:

1. Pick a position where Byte 1 has a 0 and Byte 2 has a 1. Let's use Bit 0 (the rightmost bit).
2. In Byte 1: Change Bit 0 from 0 to 1. (This adds 1 to the sum).
3. In Byte 2: Change Bit 0 from 1 to 0. (This subtracts 1 from the sum).

Modified 8 bits:

- New Byte 1: 0 1 0 1 1 1 0 1
- New Byte 2: 0 1 1 0 0 1 0 0

P₂₅

- a. In TCP, every single byte of data sent is assigned a unique sequence number, which is stored in a 4 byte field.

So, the file size L can be $2^{32} = 4,294,967,296$

b.

$$L = 4,294,967,296$$

$$\begin{aligned}\text{No. of segments} &= \left\lfloor \frac{4,294,967,296}{536} \right\rfloor \\ &= 8,012,999\end{aligned}$$

So,

each segment will take:

$$\frac{(536 + 66) \times 8}{155 \times 10^6} \text{ secs}$$

∴ 8,012,999 segments will take

$$8,012,999 \times \frac{(536 + 66) \times 8}{155 \times 10^6}$$

248.97 sec

P27.

a. seq num = 207

src port: 302

dest port: 80

b. Ack num = 207

src port: 80

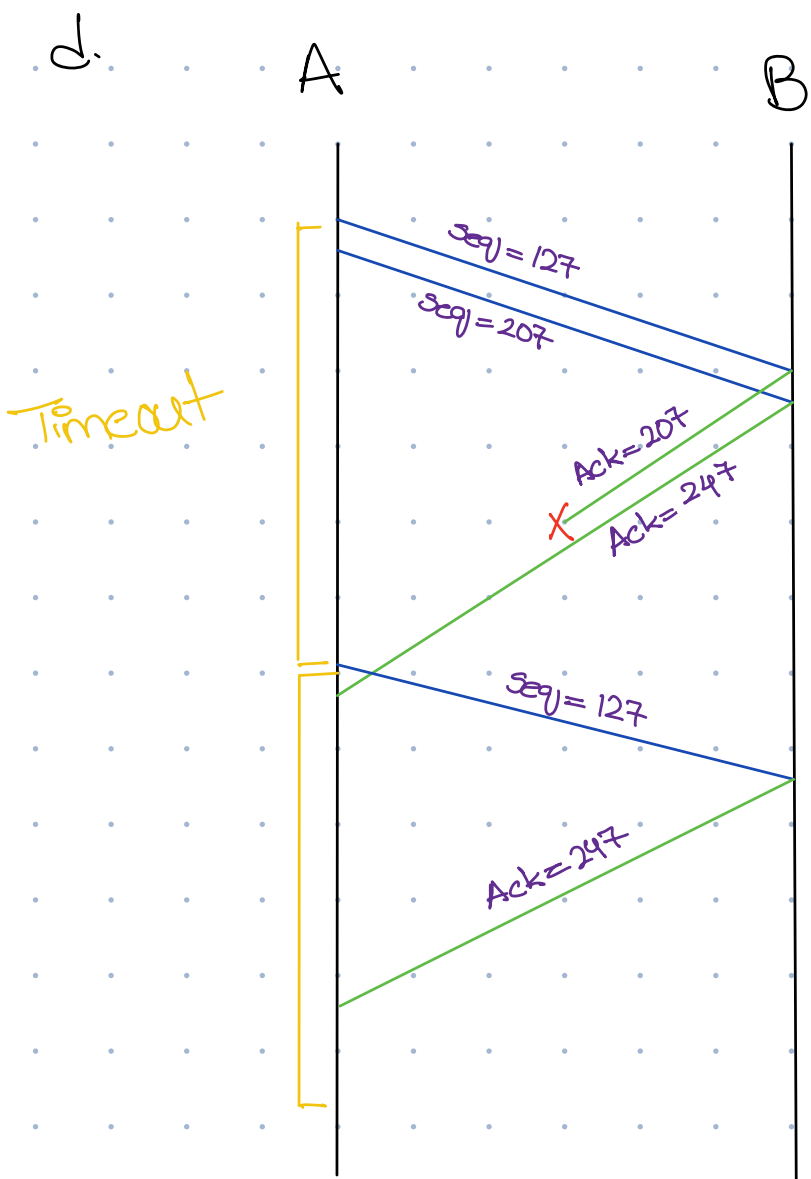
dest port: 302

c. Ack num = 127

src port: 80

dest port: 302

d.



P28. The Host A will at max send at a rate of 100Mbps as that is the link capacity. However, as Host B reads at 50Mbps, which is less than 100Mbps, its buffer will fill up. At this point, Host B will inform this to its receive window to 0 & Host A will stop sending more till receive window is increased.

P36.

TCP waits for three duplicate ACKs to distinguish between an actual dropped packet and a packet that has simply been delayed and arrived out of order.

In IP networks, segments frequently experience varying delays or take different routes, causing them to arrive at the receiver out of sequence. When a receiver gets an out-of-order segment, it immediately sends a duplicate ACK for the last in-order byte it received.

If the TCP sender performed a fast retransmit upon receiving just the first duplicate ACK, it would trigger unnecessary retransmissions for packets that were slightly delayed but still successfully en route, wasting network bandwidth.

Waiting for three duplicate ACKs serves as a heuristic. If three subsequent segments successfully arrive at the receiver while the expected segment is still missing, it provides strong statistical confidence that the missing segment was actually lost in the network rather than just temporarily delayed.

P39.

Figure 3.46(b):

No. The maximum link capacity allocated to the connection is $R/2$. As the offered load (λ'_{in}) increases beyond $R/2$, the router experiences severe congestion and frequent buffer overflows, leading to more dropped packets. Consequently, an increasing fraction of the $R/2$ bandwidth is consumed by retransmitting lost packets. Because more capacity is wasted on retransmissions, the useful throughput (λ_{out}) cannot increase beyond $R/3$ and will eventually decrease.

Figure 3.46(c):

No. The connection's maximum available share of the link capacity remains $R/2$. Under the assumption that every packet is forwarded twice on average (typically due to premature timeouts causing duplicate transmissions), the network is doing twice the work for every unit of useful data. Since the total available bandwidth is capped at $R/2$, the maximum possible rate of useful data delivery is exactly half of that capacity: $(R/2)/2 = R/4$. Therefore, λ_{out} cannot exceed $R/4$.